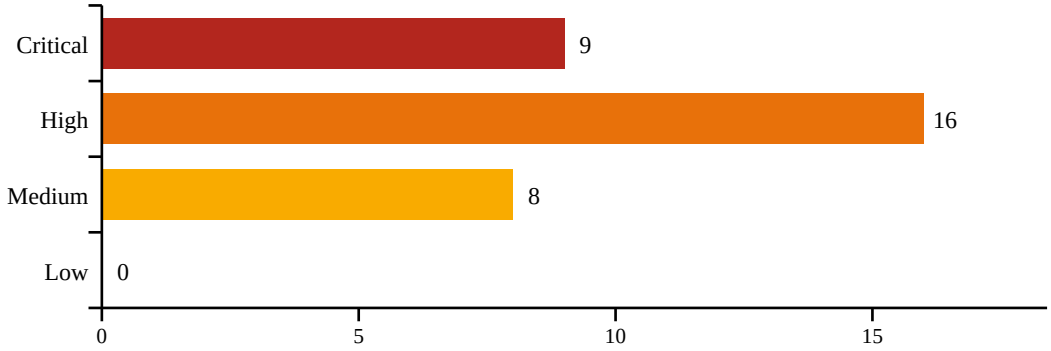
CodeMender Security Report

demo/vulnerable-app @ e299994 · 2026-10-21 13:52 UTC

100	33	11	5
Risk score / 100	Findings	With ready patch	Files · 119 lines

Findings by severity



OWASP Top 10 (2021) exposure

Category	Findings
A03:2021 Injection	10
A07:2021 Identification and Authentication Failures	6
A02:2021 Cryptographic Failures	4
A05:2021 Security Misconfiguration	4
A08:2021 Software and Data Integrity Failures	3
A01:2021 Broken Access Control	3
A06:2021 Vulnerable and Outdated Components	2
A10:2021 Server-Side Request Forgery	1

Findings overview

Sev	ID	Issue	CWE	Location	Patch
Critical	CM-e099fce6	SQL injection via string-built query	CWE-89	Vuln.java:10	-
Critical	CM-a84b2aad	Insecure deserialisation	CWE-502	Vuln.java:14	-
Critical	CM-1a0a58c0	SQL injection via string-built query	CWE-89	app.py:13	-
Critical	CM-dc9c17fb	OS command injection	CWE-78	app.py:20	-
Critical	CM-4fdb32be	OS command injection	CWE-78	app.py:26	yes
Critical	CM-dd4b4389	Insecure deserialisation	CWE-502	app.py:42	-
Critical	CM-37432e4c	Insecure deserialisation	CWE-502	app.py:43	yes
Critical	CM-e8839214	Unbounded string copy (buffer overflow)	CWE-120	parser.c:6	yes
Critical	CM-75a6dde6	Unbounded string copy (buffer overflow)	CWE-120	parser.c:8	-
High	CM-1351dae4	Hard-coded credential / API key	CWE-798	Vuln.java:7	-
High	CM-c27f213d	Weak cipher / ECB mode	CWE-327	Vuln.java:13	-

Sev	ID	Issue	CWE	Location	Patch
High	CM-f27a70a2	Hard-coded credential / API key	CWE-798	app.py:5	yes
High	CM-579ef3f4	Hard-coded credential / API key	CWE-798	app.py:6	yes
High	CM-5c1aaa98	Cross-site scripting (unsafe HTML sink)	CWE-79	app.py:32	-
High	CM-e4d69106	Path traversal in file access	CWE-22	app.py:37	-
High	CM-78121708	TLS certificate verification disabled	CWE-295	app.py:49	yes
High	CM-2372186d	Server-side request forgery (SSRF)	CWE-918	app.py:49	-
High	CM-ba162869	Format-string vulnerability	CWE-134	parser.c:7	-
High	CM-c4988d48	Hard-coded credential / API key	CWE-798	server.js:5	-
High	CM-1d1f4cfa	NoSQL / query operator injection	CWE-943	server.js:8	-
High	CM-7e0388d2	Cross-site scripting (unsafe HTML sink)	CWE-79	server.js:11	yes
High	CM-7e3cc9fa	Code injection via eval/exec	CWE-95	server.js:15	-
High	CM-5e1be7ce	Code injection via eval/exec	CWE-95	server.js:17	-
High	CM-434d1360	Path traversal in file access	CWE-22	server.js:18	-
High	CM-ef25cb99	JWT signature not verified / 'none' allowed	CWE-347	server.js:20	-
Medium	CM-44859cb4	Weak hash algorithm (MD5/SHA-1)	CWE-328	Vuln.java:12	-
Medium	CM-43cae818	Redirect to user-controlled URL	CWE-601	app.py:54	-
Medium	CM-f1c4b4b1	Insecure randomness for security token	CWE-338	app.py:58	yes
Medium	CM-5a45ae6e	Weak hash algorithm (MD5/SHA-1)	CWE-328	app.py:63	yes
Medium	CM-df0c306d	Debug mode / wildcard CORS enabled	CWE-489	app.py:67	yes
Medium	CM-c7b0349e	Container / infra runs privileged or as root	CWE-250	deployment.yaml:7	yes
Medium	CM-caed6cca	Container / infra runs privileged or as root	CWE-250	deployment.yaml:8	-
Medium	CM-9e359f62	Debug mode / wildcard CORS enabled	CWE-489	server.js:19	-

Detailed findings

[CRITICAL] SQL injection via string-built query · CWE-89 · A03:2021 Injection

Vuln.java:10 · CM-e099fce6 · confidence HIGH
```
s.executeQuery("SELECT * FROM users WHERE name = '" + user + "'");
```

Why it matters: User-influenced data is concatenated/interpolated into a SQL statement, allowing query manipulation.

Recommended fix: Use parameterised queries / prepared statements; never build SQL with string formatting.

Verification plan: Send a quote-breaking value to the affected parameter in an isolated test DB and confirm the query structure changes or errors.

[CRITICAL] Insecure deserialisation · CWE-502 · A08:2021 Software and Data Integrity Failures

Vuln.java:14 · CM-a84b2aad · confidence HIGH
```
Object read(InputStream in) throws Exception { return new ObjectInputStream(in).readObject(); }
```

Why it matters: Deserialising untrusted bytes can instantiate arbitrary objects and lead to remote code execution.

Recommended fix: Use JSON or yaml.safe_load; for Java apply ObjectInputFilter allow-lists.

Verification plan: Deserialise a benign gadget that writes a marker file in a sandbox and confirm it executes.

[CRITICAL] SQL injection via string-built query · CWE-89 · A03:2021 Injection

app.py:13 · CM-1a0a58c0 · confidence HIGH
```
cur.execute("SELECT * FROM users WHERE name = '" + name + "'")
```

Why it matters: User-influenced data is concatenated/interpolated into a SQL statement, allowing query manipulation.

Recommended fix: Use parameterised queries / prepared statements; never build SQL with string formatting.

Verification plan: Send a quote-breaking value to the affected parameter in an isolated test DB and confirm the query structure changes or errors.

[CRITICAL] OS command injection · CWE-78 · A03:2021 Injection

app.py:20 · CM-dc9c17fb · confidence HIGH
```
os.system("ping -c 1 " + host)
```

Why it matters: A shell command is built from dynamic input, enabling arbitrary command execution.

Recommended fix: Pass arguments as a list with shell=False (or execFile), validate against an allow-list.

Verification plan: In a sandbox, supply a value containing a shell metacharacter plus a harmless marker command and check whether the marker executes.

[CRITICAL] OS command injection · CWE-78 · A03:2021 Injection

app.py:26 · CM-4fdb32be · confidence HIGH
```
subprocess.call("ls " + request.args["d"], shell=True)
```

Why it matters: A shell command is built from dynamic input, enabling arbitrary command execution.

Recommended fix: Pass arguments as a list with shell=False (or execFile), validate against an allow-list.

Verification plan: In a sandbox, supply a value containing a shell metacharacter plus a harmless marker command and check whether the marker executes.

Proposed patch (for human review):
```
--- a/app.py
+++ b/app.py
@@ -24,5 +24,5 @@
 @app.route("/run")
```

```
 def run():
-    subprocess.call("ls " + request.args["d"], shell=True)
+    subprocess.call("ls " + request.args["d"], shell=False)
     return "done"
```

[CRITICAL] Insecure deserialisation · CWE-502 · A08:2021 Software and Data Integrity Failures

app.py:42 · CM-dd4b4389 · confidence HIGH

```
obj = pickle.loads(request.data)
```

Why it matters: Deserialising untrusted bytes can instantiate arbitrary objects and lead to remote code execution.

Recommended fix: Use JSON or yaml.safe_load; for Java apply ObjectInputFilter allow-lists.

Verification plan: Deserialise a benign gadget that writes a marker file in a sandbox and confirm it executes.

[CRITICAL] Insecure deserialisation · CWE-502 · A08:2021 Software and Data Integrity Failures

app.py:43 · CM-37432e4c · confidence HIGH

```
cfg = yaml.load(request.data)
```

Why it matters: Deserialising untrusted bytes can instantiate arbitrary objects and lead to remote code execution.

Recommended fix: Use JSON or yaml.safe_load; for Java apply ObjectInputFilter allow-lists.

Verification plan: Deserialise a benign gadget that writes a marker file in a sandbox and confirm it executes.

Proposed patch (for human review):

```
--- a/app.py
+++ b/app.py
@@ -41,5 +41,5 @@
 def load():
     obj = pickle.loads(request.data)
-    cfg = yaml.load(request.data)
+    cfg = yaml.safe_load(request.data)
     return str((obj, cfg))
```

[CRITICAL] Unbounded string copy (buffer overflow) · CWE-120 · A06:2021 Vulnerable and Outdated Components

parser.c:6 · CM-e8839214 · confidence HIGH

```
strcpy(buf, input);
```

Why it matters: Unbounded libc copies write past the destination buffer when input exceeds its size.

Recommended fix: Use bounded variants (snprintf, strlcpy) with explicit sizes or -fbounds-safety annotations.

Verification plan: Fuzz the entry point with oversized input under AddressSanitizer and confirm a stack/heap overflow report.

Proposed patch (for human review):

```
--- a/parser.c
+++ b/parser.c
@@ -4,5 +4,5 @@
 void handle(char *input) {
     char buf[64];
-    strcpy(buf, input);
+    strlcpy(buf, input);
     printf(input);
     sprintf(buf, "%s", input);
```

[CRITICAL] Unbounded string copy (buffer overflow) · CWE-120 · A06:2021 Vulnerable and Outdated Components

parser.c:8 · CM-75a6dde6 · confidence HIGH

```
sprintf(buf, "%s", input);
```

Why it matters: Unbounded libc copies write past the destination buffer when input exceeds its size.

Recommended fix: Use bounded variants (snprintf, strlcpy) with explicit sizes or -fbounds-safety annotations.

Verification plan: Fuzz the entry point with oversized input under AddressSanitizer and confirm a stack/heap overflow report.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication Failures

Vuln.java:7 · CM-1351dae4 · confidence MEDIUM

```
String password = "hunter2hunter2";
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] Weak cipher / ECB mode · CWE-327 · A02:2021 Cryptographic Failures

Vuln.java:13 · CM-c27f213d · confidence HIGH

```
Cipher cipher() throws Exception { return Cipher.getInstance("AES/ECB/PKCS5Padding"); }
```

Why it matters: Deprecated or structurally weak encryption leaks plaintext patterns.

Recommended fix: Use AES-GCM or ChaCha20-Poly1305 with unique nonces from a KMS-managed key.

Verification plan: Encrypt repeated plaintext blocks in a test and confirm identical ciphertext blocks appear.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication Failures

app.py:5 · CM-f27a70a2 · confidence MEDIUM

```
DB_PASSWORD = "SuperSecret123!"
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

Proposed patch (for human review):

```
--- a/app.py
+++ b/app.py
@@ -3,5 +3,5 @@

 app = Flask(__name__)
-DB_PASSWORD = "SuperSecret123!"
+DB_PASSWORD = os.environ["SECRET_FROM_ENV"]
 AWS_KEY = "AKIAABCDEFGHIJKLMNOP"
```

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication Failures

app.py:6 · CM-579ef3f4 · confidence MEDIUM

```
AWS_KEY = "AKIAABCDEFGHIJKLMNOP"
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

Proposed patch (for human review):

```
--- a/app.py
+++ b/app.py
@@ -4,5 +4,5 @@
 app = Flask(__name__)
 DB_PASSWORD = "SuperSecret123!"
-AWS_KEY = "AKIAABCDEFGHIJKLMNOP"
+AWS_KEY = os.environ["SECRET_FROM_ENV"]
```

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

app.py:32 · CM-5c1aaa98 · confidence HIGH
```
return render_template_string("<h1>Hello " + request.args["n"] + "</h1>")
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

[HIGH] Path traversal in file access · CWE-22 · A01:2021 Broken Access Control

app.py:37 · CM-e4d69106 · confidence MEDIUM
```
return send_file(os.path.join("/srv/files", request.args["f"]))
```

Why it matters: A file path is derived from request input without canonicalisation or base-directory checks.

Recommended fix: Resolve the path, verify it stays under an allowed base directory, and use an ID-to-file mapping.

Verification plan: Request a path containing parent-directory segments against a test file tree and confirm a file outside the base is returned.

[HIGH] TLS certificate verification disabled · CWE-295 · A07:2021 Identification and Authentication Failures

app.py:49 · CM-78121708 · confidence HIGH
```
return requests.get(request.args["url"], verify=False).text
```

Why it matters: Disabling certificate validation permits man-in-the-middle interception.

Recommended fix: Remove the override and trust a proper CA bundle.

Verification plan: Route the call through a proxy presenting a self-signed cert and confirm the connection succeeds.

Proposed patch (for human review):
```
--- a/app.py
+++ b/app.py
@@ -47,5 +47,5 @@
 @app.route("/fetch")
 def fetch():
-    return requests.get(request.args["url"], verify=False).text
+    return requests.get(request.args["url"], verify=True).text
```

[HIGH] Server-side request forgery (SSRF) · CWE-918 · A10:2021 Server-Side Request Forgery

app.py:49 · CM-2372186d · confidence MEDIUM
```
return requests.get(request.args["url"], verify=False).text
```

Why it matters: The server fetches a URL controlled by the caller and can be pointed at internal services or metadata endpoints.

Recommended fix: Validate scheme/host against an allow-list and block link-local/private ranges.

Verification plan: Point the parameter at a local canary listener and confirm the server connects to it.

[HIGH] Format-string vulnerability · CWE-134 · A03:2021 Injection

parser.c:7 · CM-ba162869 · confidence MEDIUM
```
printf(input);
```

Why it matters: A non-constant format string lets input read or write memory through %n / %s specifiers.

Recommended fix: Use a constant format: printf("%s", input).

Verification plan: Pass %x sequences under ASan/UBSan and confirm leaked stack content.

[HIGH] Hard-coded credential / API key · CWE-798 · A07:2021 Identification and Authentication Failures

server.js:5 · CM-c4988d48 · confidence MEDIUM
```
const API_KEY = "demo-not-a-real-key-0123456789";
```

Why it matters: A secret is embedded in source and will leak via version control, logs and build artifacts.

Recommended fix: Rotate the secret immediately; load from Secret Manager / environment at runtime.

Verification plan: Confirm the value is live using a read-only identity check in a controlled setting, then rotate.

[HIGH] NoSQL / query operator injection · CWE-943 · A03:2021 Injection

server.js:8 · CM-1d1f4cfa · confidence MEDIUM
```
db.collection("users").find(req.query).toArray((e, r) => res.json(r));
```

Why it matters: Request objects are passed directly to a database query, allowing operator injection ($ne, $gt, $where).

Recommended fix: Validate and cast types explicitly; reject objects where scalars are expected.

Verification plan: Send {"$ne": null} as a credential field in a test DB and confirm authentication bypass.

[HIGH] Cross-site scripting (unsafe HTML sink) · CWE-79 · A03:2021 Injection

server.js:11 · CM-7e0388d2 · confidence HIGH
```
document.getElementById("out").innerHTML = req.query.name;
```

Why it matters: Untrusted data reaches an HTML sink without contextual output encoding.

Recommended fix: Use textContent / auto-escaping templates and sanitise rich content with an allow-list sanitiser.

Verification plan: Submit a benign marker tag as input and confirm it is rendered unescaped in the response DOM.

Proposed patch (for human review):
```
--- a/server.js
+++ b/server.js
@@ -9,5 +9,5 @@
 });
 app.get("/greet", (req, res) => {
-  document.getElementById("out").innerHTML = req.query.name;
+  document.getElementById("out").textContent = req.query.name;
   res.send("ok");
 });
```

[HIGH] Code injection via eval/exec · CWE-95 · A03:2021 Injection

server.js:15 · CM-7e3cc9fa · confidence MEDIUM
```
exec("zip -r out.zip " + req.query.dir, () => res.send("done"));
```

Why it matters: Dynamic evaluation of strings can execute attacker-controlled code.

Recommended fix: Remove eval/exec; use ast.literal_eval or JSON parsing for data.

Verification plan: Feed an expression with an observable side-effect marker in a sandbox and confirm evaluation.

[HIGH] Code injection via eval/exec · CWE-95 · A03:2021 Injection

server.js:17 · CM-5e1be7ce · confidence MEDIUM
```
app.get("/calc", (req, res) => res.send(String(eval(req.query.expr))));
```

Why it matters: Dynamic evaluation of strings can execute attacker-controlled code.

Recommended fix: Remove eval/exec; use ast.literal_eval or JSON parsing for data.

Verification plan: Feed an expression with an observable side-effect marker in a sandbox and confirm evaluation.

[HIGH] Path traversal in file access · CWE-22 · A01:2021 Broken Access Control

server.js:18 · CM-434d1360 · confidence MEDIUM
```
app.get("/file", (req, res) => res.send(require("fs").readFileSync("./uploads/" + req.query.name)));
```

Why it matters: A file path is derived from request input without canonicalisation or base-directory checks.

Recommended fix: Resolve the path, verify it stays under an allowed base directory, and use an ID-to-file mapping.

Verification plan: Request a path containing parent-directory segments against a test file tree and confirm a file outside the base is returned.

[HIGH] JWT signature not verified / 'none' allowed · CWE-347 · A07:2021 Identification and Authentication Failures

server.js:20 · CM-ef25cb99 · confidence HIGH
```
const claims = jwt.decode(token, { algorithms: ["none"] });
```

Why it matters: Tokens are accepted without cryptographic verification, letting anyone forge identities.

Recommended fix: Always verify signature, pin algorithms and enforce expiry.

Verification plan: Submit an unsigned token with elevated claims and confirm acceptance in a test environment.

[MEDIUM] Weak hash algorithm (MD5/SHA-1) · CWE-328 · A02:2021 Cryptographic Failures

Vuln.java:12 · CM-44859cb4 · confidence HIGH
```
byte[] hash(byte[] d) throws Exception { return MessageDigest.getInstance("MD5").digest(d); }
```

Why it matters: MD5/SHA-1 are collision-prone and unsuitable for security purposes such as password storage.

Recommended fix: Use SHA-256+ for integrity and Argon2/bcrypt/scrypt for passwords.

Verification plan: Demonstrate collision/pre-image cost vs. modern hash using a test corpus; confirm use on a security path.

[MEDIUM] Redirect to user-controlled URL · CWE-601 · A01:2021 Broken Access Control

app.py:54 · CM-43cae818 · confidence MEDIUM
```
return redirect(request.args.get("next"))
```

Why it matters: Unvalidated redirect targets enable phishing via trusted domains.

Recommended fix: Allow only relative paths or an allow-list of hosts.

Verification plan: Pass an external absolute URL and confirm a 3xx to that host.

[MEDIUM] Insecure randomness for security token · CWE-338 · A02:2021 Cryptographic Failures

app.py:58 · CM-f1c4b4b1 · confidence MEDIUM
```
token = str(random.randint(100000, 999999))
```

Why it matters: A predictable PRNG generates a value that must be unguessable.

Recommended fix: Use secrets.token_urlsafe / crypto.randomBytes / SecureRandom / crypto/rand.

Verification plan: Seed-recover or enumerate generated values in a test harness to show predictability.

Proposed patch (for human review):
```
--- a/app.py
+++ b/app.py
@@ -56,5 +56,5 @@

 def make_reset_token():
```

```
-    token = str(random.randint(100000, 999999))
+    token = str(secrets.SystemRandom().choice(100000, 999999))
     return token
```

[MEDIUM] Weak hash algorithm (MD5/SHA-1) · CWE-328 · A02:2021 Cryptographic Failures

app.py:63 · CM-5a45ae6e · confidence HIGH

```
return hashlib.md5(pw.encode()).hexdigest()
```

Why it matters: MD5/SHA-1 are collision-prone and unsuitable for security purposes such as password storage.

Recommended fix: Use SHA-256+ for integrity and Argon2/bcrypt/scrypt for passwords.

Verification plan: Demonstrate collision/pre-image cost vs. modern hash using a test corpus; confirm use on a security path.

Proposed patch (for human review):

```
--- a/app.py
+++ b/app.py
@@ -61,5 +61,5 @@

 def store_password(pw):
-    return hashlib.md5(pw.encode()).hexdigest()
+    return hashlib.sha256(pw.encode()).hexdigest()
```

[MEDIUM] Debug mode / wildcard CORS enabled · CWE-489 · A05:2021 Security Misconfiguration

app.py:67 · CM-df0c306d · confidence HIGH

```
app.run(host="0.0.0.0", debug=True)
```

Why it matters: Debug consoles or permissive CORS expose internals and authenticated endpoints to any origin.

Recommended fix: Disable debug in production; restrict CORS to known origins.

Verification plan: Request an error page / cross-origin call and confirm debugger output or permissive headers are returned.

Proposed patch (for human review):

```
--- a/app.py
+++ b/app.py
@@ -65,3 +65,3 @@

 if __name__ == "__main__":
-    app.run(host="0.0.0.0", debug=True)
+    app.run(host="0.0.0.0", debug=False)
```

[MEDIUM] Container / infra runs privileged or as root · CWE-250 · A05:2021 Security Misconfiguration

deployment.yaml:7 · CM-c7b0349e · confidence HIGH

```
privileged: true
```

Why it matters: Over-privileged workload or world-open network rule increases blast radius.

Recommended fix: Drop privileges, set runAsNonRoot and restrict CIDR ranges.

Verification plan: Inspect the rendered manifest/plan with a policy engine to confirm the privileged setting.

Proposed patch (for human review):

```
--- a/deployment.yaml
+++ b/deployment.yaml
@@ -5,4 +5,4 @@
     - name: app
       securityContext:
-        privileged: true
+        privileged: false
         runAsUser: 0
```

[MEDIUM] Container / infra runs privileged or as root · CWE-250 · A05:2021 Security

Misconfiguration

deployment.yaml:8 · CM-caed6cca · confidence HIGH
```
runAsUser: 0
```

Why it matters: Over-privileged workload or world-open network rule increases blast radius.

Recommended fix: Drop privileges, set runAsNonRoot and restrict CIDR ranges.

Verification plan: Inspect the rendered manifest/plan with a policy engine to confirm the privileged setting.

[MEDIUM] Debug mode / wildcard CORS enabled · CWE-489 · A05:2021 Security

Misconfiguration

server.js:19 · CM-9e359f62 · confidence HIGH
```
app.use(require("cors")({ origin: "*" }));
```

Why it matters: Debug consoles or permissive CORS expose internals and authenticated endpoints to any origin.

Recommended fix: Disable debug in production; restrict CORS to known origins.

Verification plan: Request an error page / cross-origin call and confirm debugger output or permissive headers are returned.

Method and limits

Findings come from rule-based static analysis mapped to CWE and OWASP. Verification plans describe how a finding would be confirmed in a sandbox; no exploit was executed. Patches are suggestions that must be reviewed and tested before merge.